The tools and techniques of R programming
Lesson-12
Sorting Algorithms implemented in R

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
Email-Id:hod.ca@technoindiaeducation.com

December 27, 2021

Lesson Objectives and introduction to sorting

Objectives Of the lesson: To provide with the concepts of standard sorting Techniques.

Introduction: Explicit sorting procedures are not required in R; because R has an inbuilt **sort()** function that can be used for sorting as shown during our discussion on vectors.Still, we may like to make use of our own routines for the purpose. With this view in mind,some standard sorting algorithms have been discussed and implemented as shown below.

BUBBLE SORT

The word **'bubble'** means a globular body of liquid filled with air or gas and the **'sort'** as a noun means an arrangement of data in a chosen sequence. The chosen sequence of data may be from the smallest value to the largest in case of numeric data (or in alphabetic sequence for the alphabetic data) or it may be from the largest data value to the smallest (or in reverse alphabetic order). The first type of sequencing is called ascending order or increasing order and the other type of arranging is called descending or decreasing order. The word bubble has been associated with a particular type of arranging process in which the data values bubbles down one by one in each step to form the desired order. For example, if we select the ascending order of arrangement, then in each step of the sorting process,the largest one of the values compared will bubble down in each step and hence the name. Let us now see in steps, how the algorithm works.
Let A[1],A[2],A[3],. . . A[n] be the set of data values to be bubble-sorted, the default order being the ascending order. This means that we want to arrange the values in such a way that will make the set to possess the property as shown below:
A[1]<A[2]<A[3]. . . . . A[n-1]<A[n].

The Process of Bubble Sorting

The process:

Step1. We compare the values to see if A[1]<A[2], A[2]<A[3] and so on up to the last so that A[n-1]<A[n]. If in any of the comparisons, the order does not hold, we interchange the values to put them in the desired order. So, at the end of this step, the largest value bubbles down at the nth location. Even if we call it sinking down, the smaller values may be called bubbled up after the n comparisons.

Step2. As the value A[n] comes to its desired position, we should not disturb it. So, we compare the remaining (n-1) values similarly to get the largest one among them at the $(n-1)^{th}$ location i.e. A[n-1] contains the right value after the (n-1) comparisons. In this way, the comparisons are continued until we finish the $(n-1)^{th}$ step which is also called the $(n-1)^{th}$ pass, in which only one comparison will be done to bring the first two values in their right locations. Thus, we make (n-1)+(n-2)+(n-3)+.....3+2+1 number of comparisons to complete the sorting process. The summation of the above A.P. series gives us:

(n(n-1))/2 $=n^2/2 - n/2 = .n^2/2+$ O(n)=O(n^2)

The R script implementing the bubble sort Process

```
cat("How Many Elements to sort? ")
m=scan(n=1)
array=vector(mode="numeric",length=m)
# The above statement is to allocate m blank vector locations
for(i in 1:m){
print(paste("\n Enter the element for location number",i))
array[i]=scan(n=1)
}
# Starts Bubble sorting Process
for(i in 1:(m-1)){
for(j in 1:(m-i)){
if(array[j]>array[j+1]){
 t=array[j]
 array[j]=array[j+1]
array[j+1]=t } }}
# Displaying of sorted elements
print("\n The sorted array is: \n")
for(i in 1:m){
 print(array[i])
 }
```

Bubble sorting Using Recursion

```
 bubble=function(v){
source("Recursive_Bubble_sorting.R")
R=bubble_sort(v)
print(R)
print("Thank you")}
#Recursive_Bubble_sorting.R
bubble_sort = function(vec) {
 new_vec = swap_pass(vec)
if(isTRUE(all.equal(vec, new_vec))) {
return(new_vec)
 } else {
 return(bubble_sort(new_vec))}}
greater = function(vector_pair) {
if(vector_pair[1] >vector_pair[2]) return(TRUE) else return(FALSE)}
swap_if_greater = function(pair) {
 if(greater(vector_pair)) {
 return(rev(vector_pair))
 } else {
 return(vector_pair) }}
 swap_pass = function(vec) {
for(i in seq(1, length(vec)-1)) {
 vec[i:(i+1)] = swap_if_greater(vec[i:(i+1)]) }
```

SELECTION SORT using R

This sorting process is so named because of the method that we use. Here, we 'select' the smallest value, by default, among the n given values and then put it in the first location. In the next step we again 'select' the smallest of the remaining (n-1) values and put it in the second location. We continue this process until we put the largest value in the nth location. It is clear that the number of comparisons in selection sort will be same as that of bubble sort. So, its complexity will also be O(n^2). This algorithm is implemented below.

```
# SELECTION Sort through Iteration
cat("\n How Many Elements to sort? ")
m=scan(n=1)
array=vector(mode="numeric",length=m)
for(i in 1:m){
print(paste("\n Enter the element for location number",i))
array[i]=scan(n=1)} for(i in 1:(m-1)){
smallest=array[i]
for(j in (i+1):m){
if(smallest >array[j]){
smallest=array[j]
loc=j}}
print(smallest)
```

Performing Selection Sort in R Contd.

```
t=array[i]
array[i]=smallest
array[loc]=t}
print(m)
# Displaying of sorted elements
print("\n The sorted array is: \n")
for(i in 1:m){
print(array[i]) }
# SELECTION Sort through Recursion
selectionSort = function(arr) {
# Find the smallest value in the list (it is a sort of cheating,
 because we're using the highly optimized min() function).
smallest = min(arr)
 rest = arr[arr != smallest]
 if (length(rest) >1) {
 rest = selectionSort(arr[arr != smallest]) }
 c(smallest, rest)}
```

INSERTION SORT in R

This method of sorting is so named because it follows a rule of insertion of an element at its proper position starting from the first location.
At the outset, it is assumed that the value at the first location is trivially sorted in the desired order,ascending by default.Then A[2] is taken to compare with A[1]. The value at A[2] is placed so that A[1] and A[2] come to their sorted form i.e. A[1]<A[2], by default ordering. Then the value at A[3] is taken and compared with those in A[1] and A[2] . The value at A[3] may occupy the location of either A[1] or A[2] or it may be in its position so that the values at locations A[1],A[2] and A[3] become sorted i.e. A[1]<A[2]<A[3], assuming the ascending order of sequencing. This process is continued until the value at the nth location is put in its right location. The process of sorting is similar to the process of card sorting done by the players and is very slow.
In this sorting also, the total number of comparisons is :(n(n-1))/2 for the worst case because the number of comparisons will be 1 for the first value, 2 for the first two values and so on. Thus, we have, 1+2+3+4+. . . .+(n-1) comparisons.
However, the average number of comparisons may be calculated as under:
1/2 +2/2 +3/2 +. . . +((n-1))/2= (n(n-1))/4 =O(n^2).
The following procedure implements the insertion sort.

Insertion sort in R through iteration

```
cat("\n How Many Elements to sort? ")
m=scan(n=1)
array=vector(mode="numeric",length=m)
for(i in 1:m){
print(paste("\n Enter the element for location number",i))
array[i]=scan(n=1)}
for(i in 1:(m-1)){
for(j in (i+1):m){
if(array[i]>array[j]){
t=array[i]
array[i]=array[j]
array[j]=t}}}
print("The sorted numbers are as shown below:\n")
for(i in 1:m)
print(array[i])
```

Concepts of QuickSort

It is also sometimes called **Partition-exchange sort**. It was proposed by Sir Charles Antony Richard (Tony) Hoare in 1959/60. This elegant sorting algorithm is based on the principle of **"divide and Conquer"** which implies dividing the array to be sorted into two disjoint sublists first based on a value called the pivotal value or simply the **'pivot'**. such that all items greater than the pivot follow it, and all elements less than the pivot precede it.The process is repeated with each sub-list containing 2 or more elements and thus generating further sub-lists until we obtain a partition with one or no elements, when it becomes already sorted.The process of partitioning is recursive in nature.

The sorting process is very fast on **"random"** data, but unsuitable for mission-critical applications due to the very bad worst-case behaviour. The **'quick'ness** of the sorting process depends basically on the selection of the pivot. A wrong selection of the pivot may lead to quadratic time complexity **(O(n^2))** and the right selection equalises both sub-lists in size and leads to linear arithmetic (**"n log n"**) time complexity.

Concepts of QuickSort Contd.

Examples of some bad choice of pivot may be , the smallest one,the largest one or the middlemost one.In particular, if the smallest or the largest one is chosen, then one of the sub-lists will become empty. A randomly chosen one through a random generator is also an unwise choice.

Some people like to take the median value of the array; this is also not a good choice because it increases the computational complexity. **A good choice may be the median of the three values, the smallest,the middlemost one and the largest one from the given list**.

Let us take some illustrative example.

4,3,2,7,8,5,9,6,1,12,10

The median of (1,5,12) is 5. We take 5 as the pivot. Now, starting from the rightmost value of the list, we compare values one by one until we face the first value less than 5. At this point, we interchange the smaller value with the pivot. Thus we get here 4,3,2,7,8,1,9,6,5,12,10

We then start comparing from the leftmost value of the list one by one until we first encounter with a value which is greater than 5 and then perform the interchange as before. Thus, we get the following list:

4,3,2,5,8,1,9,6,7,12,10

Now, we again start comparing with the values from the rightmost position until

Concepts of QuickSort Contd.

we get the first value less than 5 and make the interchange as before to generate the following list:
4,3,2,1,8,5,9,6,7,12,10
Now, performing the comparison again from the leftmost position as before and making the interchange, we get,
4,3,2,1,5,8,9,6,7,12,10.
So, we get two sub-lists as shown below:
(4,3,2,1) 5 (8,9,6,7,12,10.)
Proceeding in this way recursively, we get the list sorted.

Complexity Analysis of Quick Sort: Worst Case

The Recurrence Relation is:
T(0) = T(1) = 0 (base case)
T(N) = N + T(N-1)
For solving the Recurrence Relation, we proceed as follows:
T(N) = N + T(N-1)
T(N-1) = (N-1) + T(N-2)
T(N-2) = (N-2) + T(N-3)
...
T(3) = 3 + T(2)
T(2) = 2 + T(1)
T(1) = 0
Hence,
T(N) = N + (N-1) + (N-2) ... + 3 + 2
N^2

which is O(N^2)

Recurrence Relation for quick Sort

The Recurrence Relation for the best case is:
T(0) = T(1) = 0 (base case)
T(N) = 2T(N/2) + N, when the pivot is in the middle.
To solve the Recurrence Relation, we proceed as follows:
Dividing both sides by N, we get,
T(N)/N=N/N+2T(N/2)/N
T(N/)/N=1+T(N/2)/N/2
T(N/2)/N/2=1 +T(N/4)/N/4
T(N/4)/N/4=1+T(N/8)/N/8
T(N/(N/2))/N/N/2=1+T(N/N)/N/N= 1 +T(1)/1
Or,T(2)/2=1+T(1)/1 =1. since T(1)=0
Hence, T(2)/2 =1
So, T(N)/N=1+1+. . . . . 1=log(N) terms
Thus,T(N)=Nlog(N), which is O(Nlog(N))
Now, the R script implementing the above algorithm is as shown below:

QUICKSORT —Version1

```
qs = function(vec) {
if(length(vec) >1) {
 pivot = vec[1]
low = qs(vec[vec >pivot])
mid = vec[vec == pivot]
 high = qs(vec[vec >pivot])
c(low, mid, high)}
 else
 vec
#quicksort version–2
quicksort = function(v) {
  Pick a number at random from the elements of the vector v
mid = sample(v, 1) # Here the sample function returns 1 randomly se-
lected value from the vector v
# Place-holders for left and right values.
left = c() # This is done for initialization of the vector c;
right = c()  # Initially c() contains NULL
```

QUICKSORT —Version2 Contd.

```
# We Move all the smaller values to the left, bigger values to the right.
lapply(v[v != mid], function(d) {
 if (d <mid) {
 left <<- c(left, d)
# Here, the super assignment operator is being used
 } else {
 right <<- c(right, d)
 }
 })
 if (length(left) >1) {
left = quicksort(left)
 }
 if (length(right) >1) {
right = quicksort(right)
 }
# Finally, return the sorted values.
 return( c(left, mid, right)) }
```

MERGESORT in R

The process of mergesort follows a divide and conquer policy for sorting an array of elements.The policy comprises three steps, namely,**(1) Divide, (2) Conquer and (3) Combine**. The divide process breaks a problem into multiple small subproblems recursively until the subproblems become atomic ones; the atomic subproblems can then be solved easily i.e. conquered; the small solutions can then be combined to obtain the solution of the original problem.

In mergesort, a given unsorted array of n elements is divided into n sub-arrays each containing 1 element which then becomes sorted trivially. The subarrays are then combined i.e.merged repeatedly to produce new sorted subarrays. The process is continued until the final sorted array is obtained. **The complexity of mergesort is O(n* log n) in all cases. So, it is a very efficient algorithm**. We illustrate the steps of the algorithm below with an array of 8 elements. 5 7 4 3 9 2 1 6

Step 1: Divide the array into two equal halves

5 7 4 3

9 2 1 6

MERGESORT in R–Contd.

Step 2: We divide each of them again into two equal halves:
5 7 4 3 9 2 1 6
Step 3: We divide each of the subarrays again to obtain the following:
5 7 4 3 9 2 1 6
Step 4. As we have now obtained atomic arrays, we now start combining them as shown below: 5 7 3 4 2 9 1 6 [combining in sorted order]
Step5. We combine again, to get the following:
3 4 5 7 1 2 6 9
Step 6. We now get the sorted array finally as shown below:
1 2 3 4 5 6 7 9
The process has been implemented below in R.

MERGESORT implementation in R

```
mergesort=function(A) {
if(length(A)>1) {
q = ceiling(length(A)/2)
a = mergesort(A[1:q])
 b = mergesort(A[(q+1):length(A)])
merge(a,b)
} else {
return(A) }}
merge=function(a,b) {
 l1=length(a);l2=length(b)
r=vector(mode="numeric",length=(l1+l2))
i=1; j=1; k=1;
for(k in 1:(l1+l2)) {
 if((i<=l1 && a[i]<b[j]) || j2) {
 r[k] = a[i]
 i=i+1
} else {
r[k] = b[j]
j=j+1 } }
return(r)}
```